Where Does the Time Go? Profiling and Quantizing $\pi_{0.5}$ Inference on Consumer GPUs

Issac Roy
iroy@ucsd.edu

Yash Tandon
ytandon@ucsd.edu

Anirudh Annabathula
vannabathula@ucsd.edu

Zhijian Liu
zhl209@ucsd.edu

Abstract

Vision-language-action (VLA) models such as $\pi_{0.5}$ turn camera images and a language instruction into robot actions, but a single forward pass takes tens of milliseconds even on a high-end GPU, which limits how fast a robot can react. Quantization is the obvious way to speed it up, yet recent work finds that quantized VLAs keep their task success while running no faster, or even slower, than the BF16 baseline. We argue that this gap comes from applying language-model quantization recipes without asking which resource each stage of the model is limited by. We build a roofline model of $\pi_{0.5}$ that predicts, for each stage, GPU and number format, whether the stage is limited by compute or by memory bandwidth. It predicts that the vision encoder and language model, which make up 86% of BF16 latency on an RTX 3090 Ti at 10 denoising steps and 97% at 2, are compute-bound, so weight-only quantization cannot speed them up while 8-bit activations can. In this quarter we reproduce $\pi_{0.5}$ on the LIBERO benchmark, measure its per-stage latency on an RTX 3090 Ti and an RTX 5080, compare those measurements against the roofline, and test whether simulated 8-bit and 4-bit quantization keeps task success.

Code: https://github.com/TheBoyRoy05/PiQuant

1 Introduction 2
2 Methods 4
3 Results 5
4 Discussion 6
5 Conclusion 6
References 6

1 Introduction

A VLA policy reads one or more camera images and a task instruction and outputs a short chunk of robot actions. $\pi_{0.5}$ (Physical Intelligence et al. 2025) does this in three stages. A SigLIP vision encoder turns each image into 256 tokens. A Gemma-2B language model reads the image and text tokens, the prefix, and caches its keys and values. A smaller action expert then reads that cache and produces a chunk of 50 actions by flow matching, running about 10 denoising steps. Each control step repeats the whole pass, so inference latency directly caps the control rate.

Recent work speeds this pass up by fusing kernels and by cutting the number of denoising steps. We focus on the lever with the largest headroom on consumer GPUs, and the one that has not yet delivered: quantization, storing weights, activations or both in 8 or 4 bits instead of 16. Quantization has two separate effects on a GPU. Smaller weights reduce the bytes read from memory, which helps a stage that is limited by memory bandwidth. Lower-precision activations let the matrix multiplications run on INT8 tensor cores, which helps a stage that is limited by compute. On consumer GeForce cards this matters a lot, because INT8 tensor-core math runs at four times the rate of BF16 with FP32 accumulation, the mode PyTorch uses for BF16 matrix multiplications. A recipe only speeds up a stage if it targets the resource that stage is limited by.

The roofline model (Williams, Waterman and Patterson 2009) makes this concrete. A stage takes at least as long as its arithmetic divided by peak compute, and at least as long as its memory traffic divided by memory bandwidth, whichever is larger. Table 1 applies this to $\pi_{0.5}$ on an RTX 3090 Ti using the model shapes from the openpi implementation. The vision encoder and language model are compute-bound, and they make up 86% of the 10-step BF16 latency, or 97% with 2 steps. Weight-only W4A16 quantization, the standard recipe for language models, leaves them unchanged. W8A8 cuts the language model from 46.95 to 12.96 ms. Only the action expert is memory-bound, so it is the one stage where smaller weights pay off.

Table 1: Predicted latency floor in ms for $\pi_{0.5}$ on an RTX 3090 Ti with 3 camera views (LeRobot also encodes the config's empty camera) and the prompt padded to 200 tokens. C marks a compute-bound stage and M a memory-bound one. Produced by `uv run python -m piquant.roofline`.

Format	Vision encoder	LLM	Action expert (per step)	Total, 10 steps	Total, 2 steps
BF16	8.20 (C)	46.95 (C)	0.86 (M)	63.8	56.9
W8A8	2.28 (C)	12.96 (C)	0.44 (M)	19.6	16.1
W4A16	8.20 (C)	46.95 (C)	0.48 (C)	60.0	56.1
W4A8	2.28 (C)	12.96 (C)	0.23 (M)	17.5	15.7

These are lower bounds. Real kernels add overhead, and an activation-quantized layer must also quantize its input on the fly. LeRobot also keeps the SigLIP vision encoder in FP32, which runs as TF32 at half the BF16 rate, so its as-run floor on the 3090 Ti is 16.40 ms rather than 8.20 ms. Our question for this quarter is how far measured latency sits above

these floors on consumer GPUs, and whether quantizing the compute-bound stages turns into real speedup while keeping task success.

1.1 Literature Review

VLA models. π_0 (Black et al. 2024) introduced the design $\pi_{0.5}$ builds on: a PaliGemma vision-language backbone with a separate action expert trained by flow matching. $\pi_{0.5}$ (Physical Intelligence et al. 2025) keeps that architecture and adds co-training for open-world generalization. It also conditions the action expert's normalization layers on the flow timestep, which adds weights the expert must read on every denoising step.

Where VLA inference time goes. Ma et al. (2025) optimize π_0 on an RTX 4090 down to 27.3 ms for two camera views and derive a per-GEMM roofline whose total is 19.7 ms. Our roofline reproduces their vision, language-model and action-expert floors within 5%, which we use as a check on our model. VLA-Perf (Jiang et al. 2026) measures the arithmetic intensity of each stage and finds the vision encoder (321.4 FLOP/byte) and language model (542.8) above the RTX 4090's balance point of 163.7, so compute-bound, and the action expert (54.0) below it, so memory-bound. Our roofline agrees with that split.

Fewer denoising steps. SnapFlow (Luan et al. 2026) distills $\pi_{0.5}$ into a single denoising step. On LIBERO it reaches 98.75% success against 97.75% for the 10-step teacher, and cuts latency from 274 to 83 ms on their setup. Naive one-step sampling already reaches 96.75%. Cutting steps shrinks only the action expert, so after it the vision encoder and language model, both compute-bound, dominate what remains.

Quantization methods. Most quantization work targets language models. LLM.int8() (Dettmers et al. 2022) keeps outlier features in 16 bits and the rest in INT8. SmoothQuant (Xiao et al. 2022) moves activation outliers into the weights so both can be stored as INT8 (W8A8). AWQ (Lin et al. 2023) and GPTQ (Frantar et al. 2022) quantize weights only, to 4 bits, protecting the channels that matter most or compensating rounding error with second-order information. QuaRot (Ashkboos et al. 2024) rotates weights and activations to remove outliers and reach 4 bits for weights, activations and the KV cache. Weight-only methods speed up language-model decoding because decoding one token at a time is memory-bound. That reasoning does not carry over to a VLA prefix of about a thousand tokens, which is compute-bound.

Quantizing VLAs. QuantVLA (Zhang et al. 2026) calibrates per-layer scales for post-training quantization of VLAs and reports memory savings. VLAQuantBench (Xu et al. 2026) evaluates post-training quantization of four VLAs in closed loop on LIBERO. Its accuracy experiments use simulated quantization, and its one real-kernel study, on OpenVLA-OFT's language model on an RTX 4090, finds every quantized method slower than BF16:

18.8 ms per control call for BF16 against 23.8 ms for SmoothQuant and 24.1 ms for AWQ. The authors attribute this to the cost of quantizing activations and dequantizing weights at run time, but they do not profile the kernels. So far, no work we have found reports a measured speedup from quantizing a VLA. That gap is what this project targets.

1.2 Data Description

This project uses no training data of its own. We evaluate a pretrained policy in simulation and measure its latency.

- **Model.** `lerobot/pi05_libero_finetuned`, $\pi_{0.5}$ fine-tuned on LIBERO and released through Hugging Face LeRobot. The checkpoint is about 14.5 GB.
- **Benchmark.** LIBERO (Liu et al. 2023), a set of simulated tabletop manipulation tasks built on MuJoCo. We use two of its 10-task suites, LIBERO-Spatial first and then LIBERO-Long. Each episode resets a scene from a fixed initial state, runs the policy in closed loop, and records success or failure, so the data we collect are per-episode success rates and per-call latencies.
- **Hardware.** An RTX 3090 Ti (24 GB) and an RTX 5080 (16 GB) as consumer deployment targets, an A100 for batched accuracy runs, and the National Research Platform (NRP) cluster for jobs that need more compute.

LIBERO is suited to this question because $\pi_{0.5}$ is near its success ceiling there, so any accuracy loss from quantization shows up as a drop from a known baseline. That same ceiling can also hide small losses, which we keep in mind when reading results.

2 Methods

2.1 Measuring per-stage latency

We time $\pi_{0.5}$ with `piquant.profile`, which loads `lerobot/pi05_libero_finetuned` through LeRobot 0.6.1 and runs it on one synthetic observation shaped like a LIBERO rollout's: two 256×256 camera images and a 200-token prompt. Latency does not depend on pixel or token values, so synthetic inputs time the same computation as a rollout. We wrap the vision encoder, the language-model prefix pass and each denoising step in CUDA events, read the events once after each call so that timing adds no host synchronization, and report the median and 90th percentile over 20 calls after 3 warmup calls.

We measure two modes. Eager mode runs the model as written and gives the per-stage split. Compiled mode keeps the checkpoint's `torch.compile` in max-autotune mode, which is how `lerobot-eval` runs the policy; its fused graph hides the stage boundaries, so for it we report only the total. Both modes use TF32 matrix multiplications, the setting LeRobot turns on when it compiles. We compare each stage with its roofline floor for the same workload: the 968-token prefix LeRobot actually computes, from three image slots (one of them the blank camera) and the padded prompt. The floors are BF16 except the vision

encoder's, which is TF32 because LeRobot keeps that stage in FP32.

3 Results

Table 2 shows the measured latency of each stage. On the RTX 5080 the compiled model takes 80.8 ms per call, 1.5× its 54.04 ms floor, or about 12 calls per second. The language-model prefix runs at 1.5× its floor even in eager mode, which supports the roofline's prediction that it is compute-bound, and the vision encoder runs at 2.2× its TF32 floor.

Eager denoising is far from its floor. Each step takes 14.6 ms against a 0.90 ms floor, and individual steps occasionally stall for several times the median, so the eager total varies by tens of milliseconds between runs. This points to kernel-launch overhead in the action expert's many small operations rather than to compute or memory. Compiling the model removes most of it: with the same TF32 math, the total falls from 253.1 ms in eager mode to 80.8 ms.

The action expert reads 623 MB of BF16 weights and another 466 MB of FP32 weights on every denoising step. The FP32 weights belong to the adaptive normalization layers, which depend only on the flow timestep, so they make up 43% of the weight bytes the memory-bound expert reads per step.

Table 2: Measured median latency in ms per call, over 20 calls after 3 warmup calls, against the roofline floor for the 968-token prefix LeRobot computes. Floors are BF16 except the vision encoder's, which is TF32. The compiled row is the model as `lerobot-eval` runs it; eager rows give the per-stage split. RTX 3090 Ti runs are in progress, and dashes mark values not yet measured. Produced by `uv run python -m piquant.profile`.

	RTX 5080			RTX 3090 Ti		
Stage	Measured	Floor	Ratio	Measured	Floor	Ratio
Vision encoder	25.3	11.65	2.2×	–	16.40	–
LLM prefix	50.9	33.36	1.5×	–	46.95	–
Denoising, 10 steps	173.2	9.03	19.2×	–	8.60	–
one step	14.6	0.90	16.1×	–	0.86	–
Other	3.0			–		
Total, eager	253.1	54.04	4.7×	–	71.95	–
Total, eager, 2 steps	121.2	46.81	2.6×	–	65.07	–
Total, compiled	80.8	54.04	1.5×	–	71.95	–

4 Discussion

5 Conclusion

References

Ashkboos, Saleh et al. 2024. "QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs." arXiv preprint arXiv:2404.00456. [Link]

Black, Kevin et al. 2024. "π_0: A Vision-Language-Action Flow Model for General Robot Control." arXiv preprint arXiv:2410.24164. [Link]

Dettmers, Tim et al. 2022. "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale." arXiv preprint arXiv:2208.07339. [Link]

Frantar, Elias et al. 2022. "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers." arXiv preprint arXiv:2210.17323. [Link]

Jiang, Wenqi et al. 2026. "How Fast Can I Run My VLA? Demystifying VLA Inference Performance with VLA-Perf." arXiv preprint arXiv:2602.18397. [Link]

Lin, Ji et al. 2023. "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration." arXiv preprint arXiv:2306.00978. [Link]

Liu, Bo et al. 2023. "LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning." arXiv preprint arXiv:2306.03310. [Link]

Luan, Wuyang et al. 2026. "SnapFlow: One-Step Action Generation for Flow-Matching VLAs via Progressive Self-Distillation." arXiv preprint arXiv:2604.05656. [Link]

Ma, Yunchao et al. 2025. "Running VLAs at Real-time Speed." arXiv preprint arXiv:2510.26742. [Link]

Physical Intelligence et al. 2025. "$\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization." arXiv preprint arXiv:2504.16054. [Link]

Williams, Samuel, Andrew Waterman, and David Patterson. 2009. "Roofline: An Insightful Visual Performance Model for Multicore Architectures." *Communications of the ACM* 52(4): 65–76. [Link]

Xiao, Guangxuan et al. 2022. "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models." arXiv preprint arXiv:2211.10438. [Link]

Xu, Jiuyi et al. 2026. "VLAQuantBench: Closed-Loop Evaluation of Post-Training Quantization for Vision-Language-Action Models." arXiv preprint arXiv:2609.25376. [Link]

Zhang, Jingxuan et al. 2026. "QuantVLA: Scale-Calibrated Post-Training Quantization for Vision-Language-Action Models." arXiv preprint arXiv:2602.20309. [Link]